Métodos Cuantitativos

Clase 15 - Modelos lineales generalizados

Unidad III: Regresión Logística

Introducción

En las clases anteriores, hemos aprendido:

1. **Fundamentos de probabilidad** y **distribuciones** (continuas y discretas).
2. **Regresión lineal** para modelar variables continuas, bajo supuestos de normalidad y homocedasticidad.
3. **Regresión logística** para modelar variables binarias, usando la función logit como enlace.

Introducción

- Estos modelos cubren varias situaciones importantes, pero cada uno parece "**independiente**".
- Aquí surge la pregunta:

> ¿Existe un enfoque unificador que abarque tanto la regresión lineal, la logística y otros tipos de variables (conteos, proporciones, respuestas continuas positivas, etc.)?

Introducción

- La respuesta se encuentra en los **Modelos Lineales Generalizados (GLM)**.
- Esta familia de modelos proveen un **marco general** para trabajar con diferentes distribuciones de la variable respuesta, manteniendo la estructura de un predictor lineal.

¿Qué veremos hoy?

- **Conexión con lo aprendido**
- Verosimilitud y log-verosimilitud
- ¿Por qué usar GLM?
- Principales características de los GLM
- GLM en Python

Conexión con lo aprendido

Conexión con lo aprendido

1. Probabilidad y distribuciones:

- En clases anteriores, al abordar la **teoría de la probabilidad** y **las distribuciones** (continuas: Normal, Gamma; discretas: Binomial, Poisson, etc.), vimos cómo cada familia de distribución describe el comportamiento de distintas variables aleatorias.
- En un **GLM**, se aprovechan estas mismas distribuciones para modelar la variable respuesta Y, eligiendo la que mejor describa su naturaleza (conteos, 0/1, proporciones, tiempos positivos, etc.).

Conexión con lo aprendido

2. Regresión lineal:

- Se fundamenta en la **distribución Normal** y en el **enlace identidad** ($\mu = \eta$). Es un caso **particular** dentro de los GLM (familia Gaussiana, link = identidad).
- Los supuestos de **homocedasticidad** y **normalidad de errores** resultan de asumir que $Y \sim Normal$ con varianza constante.

Conexión con lo aprendido

3. Regresión logística:

- La Regresión logística se enmarca en la familia **Binomial** con enlace **logit**. Por eso, se ve ahora como un **GLM** en el que μ (la probabilidad de éxito) se relaciona con el predictor lineal mediante la función logit.
- Metodológicamente, sigue los mismos principios de un GLM: estimación por **máxima verosimilitud**, test z para coeficientes, etc.

Conexión con lo aprendido

- Con los **GLM**, todo se integra en un **solo marco**:
  - Simplemente se elige la **distribución** (Binomial, Poisson, Gamma, etc.) y el **enlace** apropiado (log, logit, etc.) según la naturaleza de Y.

¿Qué veremos hoy?

- Conexión con lo aprendido
- **Verosimilitud y log-verosimilitud**
- ¿Por qué usar GLM?
- Principales características de los GLM
- GLM en Python

Verosimilitud y log-verosimilitud

Verosimilitud y log-verosimilitud

- Antes de adentrarnos en los **Modelos Lineales Generalizados (GLM)**, es clave comprender el concepto de verosimilitud y su versión en logaritmos (**log-verosimilitud**). Estos fundamentos permiten:

Verosimilitud y log-verosimilitud

2. **Entender el criterio de máxima verosimilitud**:

- Los GLM se ajustan típicamente buscando los parámetros que **maximizan** la verosimilitud.
- Ver cómo la log-verosimilitud convierte productos en sumas facilita el razonamiento y la implementación (p. ej., en algoritmos de optimización).

Verosimilitud y log-verosimilitud

3. **Interpretar los resultados estadísticos**:

- Conociendo la verosimilitud, es más sencillo interpretar la **razón de verosimilitudes**, los **AIC/BIC** y otros diagnósticos en la familia de los GLM.
- Además, comprender la log-verosimilitud ayuda a ver por qué los algoritmos iterativos convergen a la solución “más probable”

Verosimilitud y log-verosimilitud

¿Qué es la Verosimilitud?

- La **verosimilitud** ($L(\theta)$) se define:
  - Dada una muestra de datos $X = (x_1, x_2, \ldots, x_n)$ y un modelo con parámetros θ, la verosimilitud es el producto de las probabilidades de observar esos datos asumiendo θ.
  - Formalmente,

$$L(\theta) = P(X \mid \theta) = \prod_{i=1}^{n} f(x_i \mid \theta)$$

donde $f(\cdot \mid \theta)$ es la función de densidad o masa de probabilidad según el modelo que estemos utilizando.

Verosimilitud y log-verosimilitud

¿Qué es la Verosimilitud?

- Este concepto es una forma de preguntar:

> ¿Qué tan probables son mis datos si el modelo y sus parámetros fuesen verdaderos?”

Verosimilitud y log-verosimilitud

¿Qué es la Verosimilitud?

- Imaginemos que tenemos un modelo con ciertos parámetros (por ejemplo, θ) y observamos cuánta "*compatibilidad*" tienen esos parámetros con lo que realmente observamos en la muestra.
- Formalmente, la **verosimilitud** es el **producto** de las probabilidades (o densidades) de observar cada dato dado el modelo. Cuanto más grande sea este producto, más "probable" es que ese modelo (y sus parámetros) hayan generado tus datos.

Verosimilitud y log-verosimilitud

¿Qué es la log-verosimilitud?

Verosimilitud y log-verosimilitud

Verosimilitud en la regresión logística

1. **Contexto**:

- En la **regresión logística**, tenemos datos donde la respuesta Y puede ser 0 o 1 (por ejemplo, “no compra” vs. “compra”).
- Para cada observación i, el modelo logístico nos da una probabilidad de que $Y_i = 1$. Llamemos a esa probabilidad p_i.

Verosimilitud y log-verosimilitud

Verosimilitud en la regresión logística

2. **Qué es la verosimilitud**:

- Básicamente, la verosimilitud responde a: "**¿Con estos parámetros, cuán compatibles son mis datos (los ceros y unos) con lo que el modelo predice?**"
  - Si para la observación i el modelo dice que p_i es la probabilidad de $Y_i = 1$, la probabilidad de observar Y_i en la realidad se expresa como:
- $p_i^{Y_i}(1 - p_i)^{(1-Y_i)}$
  - Si $Y_i = 1$, esta expresión se vuelve p_i.
  - Si $Y_i = 0$, se vuelve $1 - p_i$.
  - Para **toda** la muestra, multiplicamos estas probabilidades de cada observación i. Ese producto total es la **verosimilitud** (L).

Verosimilitud y log-verosimilitud

Verosimilitud en la regresión logística

3. **Ejemplo simple**:

- Supongamos que en tres observaciones, $(Y_1, Y_2, Y_3) = (1, 0, 1)$.
- Las probabilidades que el modelo nos entrega son (p_1, p_2, p_3).
- La verosimilitud es:

$$L = p_1^1(1-p_1)^0 \cdot p_2^0(1-p_2)^1 \cdot p_3^1(1-p_3)^0 = p_1 \cdot (1-p_2) \cdot p_3$$

Verosimilitud y log-verosimilitud

Log-verosimilitud en la regresión logística

1. **Por qué tomar logaritmos**:

- Multiplicar muchas probabilidades (valores entre 0 y 1) da números muy pequeños.
- Pasar a logaritmos convierte esa multiplicación en una suma de log-probabilidades. Así evitamos problemas de **subflujo** numérico y hacemos más fácil el cálculo.

Verosimilitud y log-verosimilitud

Log-verosimilitud en la regresión logística

2. **Continuando con el ejemplo anterior**:

- Tomando el $\log$ de la verosimilitud anterior:
  - $\log L \;=\; \log(p_1) + \log(1 - p_2) + \log(p_3)$
- Entonces, para n observaciones, la **log-verosimilitud** (ℓ) es la suma:
  - $\ell(\beta) = \sum_{i=1}^{n} \left[Y_i \log(p_i) + (1 - Y_i)\ \log(1 - p_i) \right],$
  - donde p_i depende de los parámetros β (coeficientes del modelo logístico).

Verosimilitud y log-verosimilitud

Log-verosimilitud en la regresión logística

3. **Maximizar la verosimilitud = Maximizar la log-verosimilitud**:

- Es **exactamente lo mismo** maximizar L o $\log(L)$, porque la función log es creciente.
- En la práctica, es mucho más cómodo y estable computar la **log-verosimilitud**.
- Al buscar los β que maximizan $\ell(\beta)$, encontramos la estimación de los coeficientes más "coherente" con los datos observados en el sentido de la probabilidad.

Verosimilitud y log-verosimilitud

Uso en modelos (por ej., GLM)

- En la práctica, se suele hablar de la verosimilitud negativa (o de $-2 \times \text{log-verosimilitud}$), porque muchos algoritmos de optimización buscan **minimizar** funciones (en vez de maximizarlas).
- Al estimar parámetros por **Máxima Verosimilitud**, se busca el $\hat{\theta}$ que maximiza $\ell(\theta)$.

Verosimilitud y log-verosimilitud

Uso en modelos (por ej., GLM)

- En resumen, la **log-verosimilitud** es la **herramienta fundamental** para estimar parámetros mediante **Máxima Verosimilitud** en modelos estadísticos (incluidos los **Modelos Lineales Generalizados**).
- Ya que transforma un producto (potencialmente muy grande o muy pequeño) en una suma, permitiendo un **tratamiento computacional y analítico más sencillo**.

¿Qué veremos hoy?

- Conexión con lo aprendido
- Verosimilitud y log-verosimilitud
- **¿Por qué usar GLM?**
- Principales características de los GLM
- GLM en Python

¿Por qué usar GLM?

¿Por qué usar GLM?

1. Modelar distintos tipos de respuesta

- **Distintos tipos de de variables**:
  - La Regresión lineal solo es apropiada cuando Y **es continua**, normalmente distribuida (con varianza constante).
  - Pero en la práctica, frecuentemente se encuentran:
  - **Datos binarios** (compra/no compra, gana/pierde, aprueba/no aprueba).
  - **Conteos** (número de clientes, accidentes, fallas de máquina).
  - **Tiempos o costos** (positivos), a menudo con distribución asimétrica (Gamma, Exponencial, etc.).

¿Por qué usar GLM?

1. Modelar distintos tipos de respuesta

- **Familias soportadas**:
  - **Binomial** (incluye Bernoulli como caso particular).
  - **Poisson** (distribución para conteos).
  - **Gamma, Inverse Gaussian** (para respuestas continuas positivas).
  - **Normal** (para respuestas continuas “clásicas”).
  - Y otras dentro de la **familia exponencial**.

¿Por qué usar GLM?

2. Unificación conceptual

- La regresión lineal y logística son **casos particulares** de la misma fórmula:

$$g(\mu) = \beta_0 + \beta_1 X_1 + \cdots + \beta_k X_k$$

donde g es la función de enlace y $\mu = E[Y]$.

- Cambiando la **función de enlace** g y la **distribución** de la respuesta, abarcamos un amplio espectro de situaciones sin "reinventar" la teoría.

¿Por qué usar GLM?

3. Flexibilidad en la práctica

- En un mismo marco, podemos abordar problemas con respuestas muy diversas sin tener que “inventar” un modelo distinto cada vez.
- Cada **distribución** (Normal, Poisson, Binomial, Gamma) tiene su **enlace canónico** (identidad, log, logit, inverso), permitiendo modelar de manera consistente.

¿Qué veremos hoy?

- Conexión con lo aprendido
- Verosimilitud y log-verosimilitud
- ¿Por qué usar GLM?
- **Principales características de los GLM**
- GLM en Python

Principales características de los GLM

Principales características de los GLM

1. Familia exponencial

- La variable respuesta Y se asume provenir de una distribución de la **familia exponencial** (e.g., Gaussiana, Binomial, Poisson, Gamma, etc.).
- Cada una define cómo se comporta la varianza en función de la media μ.

Principales características de los GLM

2. Predictor lineal

$$\eta = \beta_0 + \beta_1 X_1 + \cdots + \beta_k X_k$$

- Similar a la regresión lineal "clásica", salvo que la salida η ya no es la variable respuesta directa, sino una transformación de su media μ.

Principales características de los GLM

2. Predictor lineal

- **Interpretación de coeficientes**:
- La interpretación de los coeficientes del GLM depende del enlace, por ejemplo:
  - En enlace log (Poisson, Gamma), un incremento de unidad en X_j multiplica la media de Y por $\exp(\beta_j)$.
  - En enlace logit (Binomial), β_j indica el efecto en el log-odds de μ.

Principales características de los GLM

3. Función de enlace

$$g(\mu) = \eta \quad \Longleftrightarrow \quad \mu = g^{-1}(\eta)$$

- Conecta la media de la variable con la combinación lineal de predictores.
- Ejemplos:
  - Identidad ($g(\mu) = \mu$) para la familia Normal (regresión lineal).
  - Logit ($g(\mu) = \ln\left(\frac{\mu}{1-\mu}\right)$) para la familia Binomial (regresión logística).
  - Log ($g(\mu) = \ln(\mu)$) para la familia Poisson (conteos).

Principales características de los GLM

4. Estimación por máxima verosimilitud

- A diferencia de la mínima suma de cuadrados (regresión lineal), en GLM se optimiza la **log-verosimilitud**.
- Se usan métodos iterativos (ej., *Iterative Reweighted Least Squares (IRLS)*), donde en cada iteración se actualiza β hasta convergencia del algoritmo.

Principales características de los GLM

5. Diagnósticos y validación

- Se siguen revisando residuos, AIC, BIC y, en muchos casos, se pueden emplear métodos de validación cruzada.
- Hay que prestar atención a aspectos específicos (por ej. sobre/infra-dispersión en Poisson).

¿Qué veremos hoy?

- Conexión con lo aprendido
- Verosimilitud y log-verosimilitud
- ¿Por qué usar GLM?
- Principales características de los GLM
- **GLM en Python**

GLM en Python

GLM en Python

- Para ajustar GLM en Python, podemos usar la librería `statsmodels`, que ofrece la clase GLM y las familias correspondientes.
- Algunas de las familias provistas en `sm.families` son:
  - `sm.families.Poisson()`
  - `sm.families.Binomial()`
  - `sm.families.Gamma()`
  - etc.

GLM en Python

Regresión Poisson para conteos

- Supongamos que contamos con datos que registran la cantidad de clientes que visitan una tienda.
- Para comprender mejor este fenómeno, disponemos de distintas variables que podrían influir en la afluencia de público, tales como la temperatura, la condición de fin de semana y si el día es festivo.

	temp	fds	festivo	clientes
0	17.490802	0	0	42
1	29.014286	0	0	76
2	24.639879	0	0	67
3	21.973170	0	0	68
4	13.120373	1	0	71

GLM en Python

Regresión Poisson para conteos

Creamos el modelo GLM con familia Poisson

```python
import statsmodels.api as sm

y = df["clientes"] # vector objetivo
X = df[["temp", "fds", "festivo"]] # matriz de atributos

# añadimos intercepto
X = sm.add_constant(X)

# creamos el modelo GLM, indicando la familia a que pertenece
modelo_poisson = sm.GLM(
  y,
  X,
  family=sm.families.Poisson()
)

# generamos los resultados del modelo ajustado
resultados = modelo_poisson.fit()
```

GLM en Python

Regresión Poisson para conteos

- Reportamos los resultados:

```python
print(resultados.summary())
```

```
                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:               clientes   No. Observations:                  300
Model:                            GLM   Df Residuals:                      296
Model Family:                 Poisson   Df Model:                            3
Link Function:                    Log   Scale:                          1.0000
Method:                          IRLS   Log-Likelihood:                -1074.5
Date:                Sun, 08 Mar 2026   Deviance:                       332.76
Time:                        14:20:27   Pearson chi2:                     333.
No. Iterations:                     4   Pseudo R-squ. (CS):             1.000
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const          2.9997      0.028    107.981      0.000       2.945       3.054
temp           0.0505      0.001     42.523      0.000       0.048       0.053
fds            0.6893      0.014     50.699      0.000       0.663       0.716
festivo        0.4417      0.036     12.398      0.000       0.372       0.512
==============================================================================
```

GLM en Python

Regresión Poisson para conteos

- Con el modelo ajustado, nos gustaría estimar que sucede si:

> "Si mañana se espera una **temperatura de 25°C**, es **fin de semana** (sábado) y **no** es festivo, ¿cuántos clientes esperamos que lleguen a la tienda?"

```python
import pandas as pd

# creamos un DataFrame con las columnas exactamente en el mismo orden que X
# recordar que en X: 'const', 'temp', 'fds', 'festivo'
registro = pd.DataFrame({
    'const': [1],       # intercepto
    'temp': [25.0],     # 25°C
    'fds': [1],     # 1 => es fin de semana
    'festivo': [0]      # 0 => no es festivo
})

# ahora predecimos con nuestro modelo
pred = resultados.predict(registro)
print(pred)
```

```
0    141.395202
dtype: float64
```

GLM en Python

Regresión Poisson para conteos

```
print(pred)
```

```
0    141.395202
dtype: float64
```

- En base al resultado anterior podríamos indicar que:
  - Se estima que, bajo las condiciones indicadas (25°C, fin de semana, no festivo), la media esperada de clientes sea de **141**.

GLM en Python

Regresión Binomial

- Supongamos que administramos campañas de marketing digital y registramos cuántos **leads** (clientes potenciales) se **convirtieron** (compraron el producto) de un total de **intentos**.
- Cada fila representa una campaña con:
  1. **budget** (cantidad de dinero gastado en la campaña).
  2. **region** (por ejemplo, 0 = local, 1 = internacional).
  3. **n_leads** (número de leads contactados en la campaña).
  4. **converted** (número de leads que efectivamente compraron).

GLM en Python

Regresión Binomial

- Queremos modelar $\text{converted} \sim \text{Binom}(\text{n_leads}, p)$, donde p se relaciona logísticamente con el predictor lineal.

```python
import numpy as np
import pandas as pd
import statsmodels.api as sm

np.random.seed(42)

n = 300  # número de campañas

# Variables explicativas
budget = np.random.uniform(1000, 5000, size=n)  # presupuesto en dólares
region = np.random.choice([0, 1], size=n, p=[0.6, 0.4])  # 0=local, 1=internacional

# Número de leads contactados en cada campaña (entre 20 y 100)
n_leads = np.random.randint(20, 101, size=n)

# Coeficientes "reales"
beta_0 = -2.0
beta_1 = 0.0005   # efecto del budget
beta_2 = 0.8      # efecto de region=1

# Predictor lineal y probabilidad (enlace logit)
eta = beta_0 + beta_1 * budget + beta_2 * region
```

```
# Para cada campaña, generamos 'converted' ~ Binomial(n_leads, p)
```

	budget	region	n_leads	converted
0	2498.160475	0	24	11
1	4802.857226	0	61	36
2	3927.975767	0	80	33
3	3394.633937	1	41	28
4	1624.074562	1	40	17

GLM en Python

Regresión Binomial

Creamos el modelo GLM con familia Binomial:

```python
import statsmodels.api as sm

X = df_binomial[["budget", "region"]] # matriz de atributos
# añadimos intercepto
X = sm.add_constant(X)

# dado que es una distribución binomial, creamos el vector objetivo
# con la forma [exitos, fracasos]
exitos = df_binomial["converted"]
fracasos = df_binomial['n_leads'] - df_binomial['converted']
y = np.column_stack((exitos, fracasos))  # vector objetivo

# creamos el modelo GLM, indicando la familia a que pertenece
modelo_binom = sm.GLM(
  y,
  X,
  family=sm.families.Binomial()
)
```

GLM en Python

Regresión Binomial

- Reportamos los resultados:

```python
print(resultados_binom.summary())
```

```
                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:           ['y1', 'y2']   No. Observations:                  300
Model:                            GLM   Df Residuals:                      297
Model Family:                Binomial   Df Model:                            2
Link Function:                  Logit   Scale:                          1.0000
Method:                          IRLS   Log-Likelihood:                -806.46
Date:                Sun, 08 Mar 2026   Deviance:                       328.58
Time:                        14:20:27   Pearson chi2:                     325.
No. Iterations:                     4   Pseudo R-squ. (CS):             0.9975
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const         -2.0429      0.050    -40.472      0.000      -2.142      -1.944
budget         0.0005   1.43e-05     35.943      0.000       0.000       0.001
region         0.7556      0.033     23.083      0.000       0.691       0.820
==============================================================================
```

GLM en Python

Regresión Binomial

- Supongamos el departamento de Marketing nos solicita estimar la siguiente situación:

> "Si planeamos una nueva campaña con un **presupuesto de 4500 dólares**, dirigida a un **mercado internacional** (region=1), y se contactarán **80 leads**, ¿cuántos conversiones esperamos?"

GLM en Python

Regresión Binomial

```python
import pandas as pd

registro_bin = pd.DataFrame({
    'const': [1],
    'budget': [4500],
    'region': [1]
})

# predecimos la probabilidad de conversión (p)
p_estimada = resultados_binom.predict(registro_bin)

# p_estimada es la probabilidad promedio de que un lead se convierta,
# por lo tanto, el número esperado de éxitos en 80 leads es:
leads_nuevos = 80
conversiones_esperadas = p_estimada * leads_nuevos

print("Probabilidad de conversión estimada:", p_estimada[0])
print("Número esperado de conversiones:", conversiones_esperadas[0])
```

```
Probabilidad de conversión estimada: 0.7355931574976958
Número esperado de conversiones: 58.84745259981567
```

Regresión Binomial

```python
print("Probabilidad de conversión estimada:", p_estimada[0])
print("Número esperado de conversiones:", conversiones_esperadas[0])
```

```
Probabilidad de conversión estimada: 0.7355931574976958
Número esperado de conversiones: 58.84745259981567
```

- `p_estimada` es la probabilidad de éxito **por lead** en esa campaña.
- Multiplicar `p_estimada` * 80 nos da la **expectativa** de cuántos leads se convertirán (58 clientes).

GLM en Python

Familia Gamma

- Imaginemos que disponemos de **registros** de varias máquinas en una planta industrial, con el **tiempo (en días)** hasta que cada máquina falló, y algunos **factores** que podrían influir:
  1. **usage**: número de horas diarias de uso.
  2. **environment**: 0 si el entorno es moderado, 1 si el entorno es agresivo (mucho polvo, vibraciones, etc.).
  3. **operator_exp**: años de experiencia del operador principal (una mayor experiencia podría implicar un uso más cuidadoso).
- La variable respuesta, **time_to_fail**, es un tiempo de falla continuo y positivo. Su distribución se modela como Gamma con enlace log.

GLM en Python

Familia Gamma

Para este problema, contamos con los siguientes datos:

```python
import numpy as np
import pandas as pd
import statsmodels.api as sm

np.random.seed(99)

n = 300  # número de máquinas registradas

# 1) Horas de uso al día (uso alto -> falla más rápida)
usage = np.random.uniform(3, 12, size=n)

# 2) Entorno (0=moderado, 1=agresivo)
environment = np.random.choice([0, 1], size=n, p=[0.6, 0.4])

# 3) Experiencia del operador (entre 1 y 5 años)
operator_exp = np.random.uniform(1, 5, size=n)

# Coeficientes "reales" (en la escala log de la media)
```

	usage	environment	operator_exp	time_to_fail
0	9.050507	0	4.143134	38.259942
1	7.392706	1	3.678196	22.026383
2	10.429457	0	1.664249	1.197747
3	3.283017	1	3.556791	7.184497
4	10.272450	0	4.206369	11.358493

GLM en Python

Familia Gamma

Creamos el modelo GLM con familia Gamma:

```python
import statsmodels.api as sm

y = df_fail["time_to_fail"] # vector objetivo
X = df_fail[['usage', 'environment', 'operator_exp']] # matriz de atributos

# añadimos intercepto
X = sm.add_constant(X)

# creamos la familia Gamma con enlace log
familia_gamma = sm.families.Gamma(link=sm.families.links.log())

# creamos el modelo GLM, indicando la familia a que pertenece
modelo_gamma = sm.GLM(
  y,
  X,
  family=familia_gamma
)

```

GLM en Python

Familia Gamma

- Reportamos los resultados:

```python
print(resultados_gamma.summary())
```

```
                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:           time_to_fail   No. Observations:                  300
Model:                            GLM   Df Residuals:                      296
Model Family:                   Gamma   Df Model:                            3
Link Function:                    log   Scale:                          0.97235
Method:                          IRLS   Log-Likelihood:                -1116.5
Date:                Sun, 08 Mar 2026   Deviance:                       271.97
Time:                        14:20:27   Pearson chi2:                     288.
No. Iterations:                     11   Pseudo R-squ. (CS):            0.09683
Covariance Type:            nonrobust
===================================================================================
                    coef    std err          z      P>|z|      [0.025      0.975]
-----------------------------------------------------------------------------------
const             3.2434      0.224     14.451      0.000       2.804       3.683
usage            -0.0880      0.022     -3.977      0.000      -0.131      -0.045
environment      -0.4177      0.116     -3.589      0.000      -0.646      -0.190
operator_exp      0.1002      0.051      1.974      0.048       0.001       0.200
===================================================================================
```

- Un **coeficiente negativo** en `usage` y `environment` sugiere que, cuanto más se use la máquina o más agresivo sea el entorno, menor será el tiempo de vida.
- Un **coeficiente positivo** en `operator_exp` prolonga la vida útil al aumentar la media de la distribución Gamma.

GLM en Python

Familia Gamma

- Supongamos que desde el área de mantenimiento preventivo nos solicitan lo siguiente:

> Necesitamos estimar el tiempo de falla para una máquina que se usará 8 horas/día, en un entorno agresivo (`environment=1`), con un operador de 3 años de experiencia (`operator_exp=3`).

GLM en Python

Familia Gamma

```python
import pandas as pd

registro_fail = pd.DataFrame({
    'const': [1],
    'usage': [8],
    'environment': [1],
    'operator_exp': [3]
})

# predecimos la media de tiempo de falla
tiempo_esperado_falla = resultados_gamma.predict(registro_fail)

print("Tiempo esperado de falla (días):", tiempo_esperado_falla[0])
```

```
Tiempo esperado de falla (días): 11.271033147892243
```

- El valor retornado indica los **días promedio** que la máquina duraría antes de fallar bajo esas condiciones de uso.

Conclusiones

Conclusiones

- Los Modelos Lineales Generalizados (*Generalized Linear Models, GLM*) se han presentado como un **marco unificador** que abarca la regresión lineal clásica (Normal con enlace identidad), la logística (Binomial con enlace logit), la Poisson (conteos), la Gamma (costos o tiempos positivos), entre otras.
- A lo largo de esta sesión, hemos revisado sus componentes principales:
  1. **Familia Exponencial**: Permite seleccionar la distribución adecuada (Binomial, Poisson, Gamma, etc.) para la variable respuesta en función de sus características (binaria, conteos, continua positiva).
  2. **Función de Enlace**: Conecta la media de la respuesta (μ) con un **predictor lineal** ($\beta_0 + \beta_1 X_1 + \ldots$) mediante funciones como log, logit, identidad, etc.
  3. **Máxima Verosimilitud y Log-Verosimilitud**: Explican cómo se estiman los parámetros de manera estadísticamente coherente, transformando productos en sumas y facilitando el uso de algoritmos iterativos (IRLS).